

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Mandeep Singh(1BF24CS166)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING in COMPUTER SCIENCE AND
ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING (Autonomous Institution under
VTU) BENGALURU-560019
February to June 2026**

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum) Department
of Computer Science and Engineering**



CERTIFICATE

This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **Mandeep Singh(1BF24CS166)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Proff. Pallavi CV
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	04
2	Implement Johnson Trotter algorithm to generate permutations.	09
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	12
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	17
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	20
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	23
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	27
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	29 31
9	Implement Fractional Knapsack using Greedy technique.	34
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	37
11	Implement "N-Queens Problem" using Backtracking.	39

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab Program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

Code:

```
#include <stdio.h>

#define MAX 10

int main() {
    int n;
    int graph[MAX][MAX];
    int indegree[MAX] = {0};
    int visited[MAX] = {0};

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);

            if (graph[i][j] == 1)
                indegree[j]++;
        }
    }

    printf("\nTopological Ordering:\n");

    for (int count = 0; count < n; count++) {

        for (int i = 0; i < n; i++) {

            if (indegree[i] == 0 && !visited[i]) {

                printf("%d ", i);
                visited[i] = 1;

                // Remove outgoing edges
                for (int j = 0; j < n; j++) {
                    if (graph[i][j] == 1)
                        indegree[j]--;
                }

                break;
            }
        }
    }
}
```

```
    }  
  }  
}  
  
printf("\n");  
  
return 0;  
}
```

Output:

OutputClear

```
Enter number of vertices: 3  
Enter adjacency matrix:  
0 1 0  
0 0 0  
1 1 0  
  
Topological Ordering:  
2 0 1  
  
=== Code Execution Successful ===
```

LeetCode Problem 207:

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you must take course b_i first if you want to take course a_i.

- For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return true if you can finish all courses. Otherwise, return false.

Code:

```
#include <stdlib.h>

#include <stdbool.h>

bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int *indegree = (int *)calloc(numCourses, sizeof(int));
    int **adj = (int **)malloc(numCourses * sizeof(int *));
    int *adjSize = (int *)calloc(numCourses, sizeof(int));
    for (int i = 0; i < numCourses; i++) {
        adj[i] = (int *)malloc(numCourses * sizeof(int));
    }
    for (int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        adj[b][adjSize[b]++] = a;
        indegree[a]++;
    }
    int *queue = (int *)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }
```

```

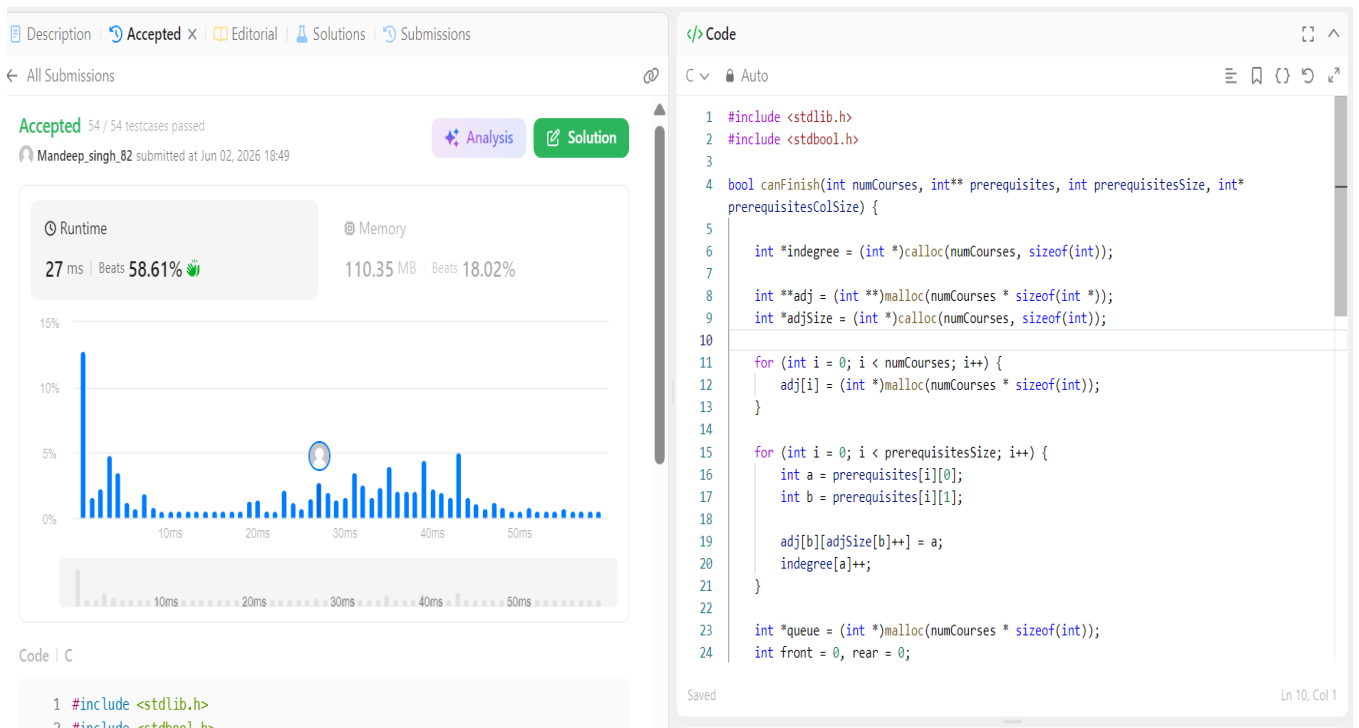
    }

    int count = 0;
    while (front < rear) {
        int course = queue[front++];
        count++;
        for (int i = 0; i < adjSize[course]; i++) {
            int next = adj[course][i];
            indegree[next]--;
            if (indegree[next] == 0) {
                queue[rear++] = next;
            }
        }
    }

    for (int i = 0; i < numCourses; i++) {
        free(adj[i]);
    }
    free(adj);
    free(adjSize);
    free(indegree);
    free(queue);
    return count == numCourses;
}

```

ScreenShot from LeetCode:



Lab Program 2:

Implement Johnson Trotter algorithm to generate permutations.

Code:

```
#include <stdio.h>

#define MAX 10

#define L -1

#define R 1

void print(int a[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int main() {
    int n, a[MAX], dir[MAX];
    printf("Enter n: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        a[i] = i + 1;
        dir[i] = L;
    }
    print(a, n);
    while (1) {
        int mobile = 0, pos = -1;
        for (int i = 0; i < n; i++) {
            if (dir[i] == L && i > 0 &&
                a[i] > a[i-1] && a[i] > mobile) {
                mobile = a[i];
                pos = i;
            }
        }
    }
}
```

```

    if (dir[i] == R && i < n-1 &&
        a[i] > a[i+1] && a[i] > mobile) {
        mobile = a[i];
        pos = i;
    }
}

if (mobile == 0)
    break;

int next = pos + dir[pos];
int t = a[pos];
a[pos] = a[next];
a[next] = t;
t = dir[pos];
dir[pos] = dir[next];
dir[next] = t;

for (int i = 0; i < n; i++)
    if (a[i] > mobile)
        dir[i] *= -1;

print(a, n);
}

return 0;
}

```

Output:

```
Output Clear  
Enter n: 3  
1 2 3  
1 3 2  
3 1 2  
3 2 1  
2 3 1  
2 1 3  
  
=== Code Execution Successful ===
```

Lab Program:3

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

Code:

```
#include <stdio.h>

#include <time.h>

#define MAX 100000

void merge(int a[], int low, int mid, int high)
{
    int i = low, j = mid + 1, k = low;
    int temp[MAX];
    while (i <= mid && j <= high)
    {
        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }
    while (i <= mid)
        temp[k++] = a[i++];
    while (j <= high)
        temp[k++] = a[j++];
    for (i = low; i <= high; i++)
        a[i] = temp[i];
}

void mergeSort(int a[], int low, int high)
{
    if (low < high)
    {

```

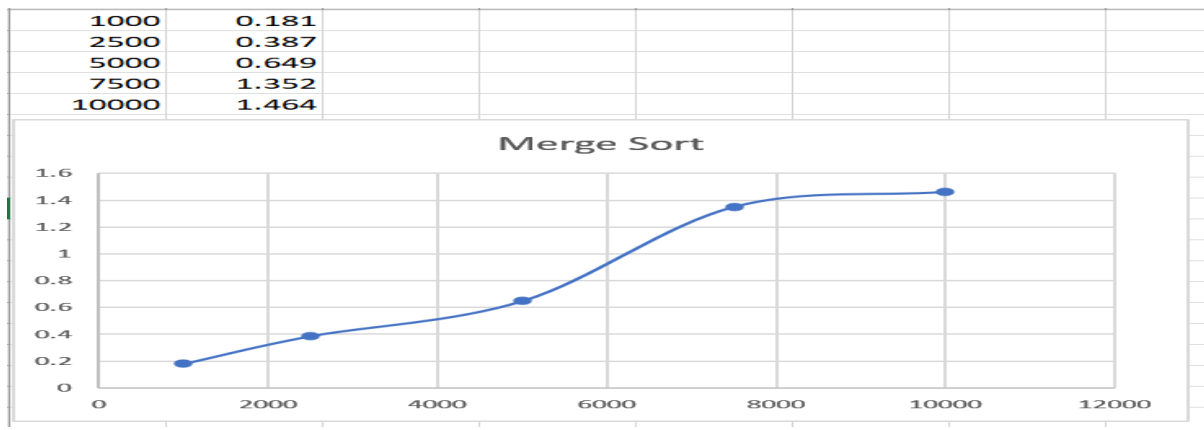
```

        int mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}

int main()
{
    int a[MAX], n, i;
    clock_t start, end;
    double time_taken;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    start = clock();
    mergeSort(a, 0, n - 1);
    end = clock();
    time_taken = ((double)(end - start)*1000.0) / CLOCKS_PER_SEC;
    printf("\nSorted array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n\nTime taken = %f seconds\n", time_taken);
    return 0;
}

```

Output:



Enter number of elements: 5

Enter elements:

41 5 78 49 30

Sorted array:

5 30 41 49 78

Time taken = 0.014000 seconds

LeetCode Problem 21:

You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one **sorted** list. The list should be made by splicing together the nodes of the first two lists.

Return *the head of the merged linked list*.

Code:

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * };
 */

struct ListNode* mergeTwoLists(struct ListNode* list1, struct ListNode* list2) {
    struct ListNode *temp=list1;
    struct ListNode *i,*j;
    int temp2;
    if(list1==NULL){
        list1=list2;
        return list1;
    }
    while(temp->next != NULL){
        temp=temp->next;
    }
    temp->next=list2;
    for(i=list1;i!=NULL;i=i->next){
        for(j=i->next;j!=NULL;j=j->next){
```

```

        if(i->val>j->val){
            temp2=j->val;
            j->val=i->val;
            i->val=temp2;
        }
    }
}

return list1;
}

```

ScreenShot from Leetcode:

The screenshot displays a LeetCode submission for the 'Merge Two Sorted Lists' problem. The submission is marked as 'Accepted' with 208/208 test cases passed. The runtime is 0 ms, beating 100.00% of other submissions, and the memory usage is 10.78 MB, also beating 100.00%. A bar chart shows the runtime performance relative to other users. The code is written in C and defines a singly-linked list structure and a function to merge two sorted lists.

Runtime Performance:

- Runtime: 0 ms | Beats 100.00%
- Memory: 10.78 MB | Beats 100.00%

Code (C):

```

1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     struct ListNode *next;
6  * };
7  */
8 struct ListNode* mergeTwoLists(struct ListNode* list1, struct ListNode* list2) {
9     struct ListNode *temp=list1;
10    struct ListNode *i,*j;
11    int temp2;
12    if(list1==NULL){
13        list1=list2;
14        return list1;
15    }
16
17    while(temp->next != NULL){
18        temp=temp->next;
19    }
20    temp->next=list2;
21    for(i=list1;i!=NULL;i=i->next){
22        for(j=i->next;j!=NULL;j=j->next){
23            if(i->val>j->val){
24                temp2=j->val;

```


Lab Program:4

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[low];
    int i = low;
    int j = high;

    while (i < j) {

        while (arr[i] <= pivot && i <= high - 1) {
            i++;
        }

        while (arr[j] > pivot && j >= low + 1) {
            j--;
        }

        if (i < j) {
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[low], &arr[j]);
    return j;
}

void quickSort(int arr[], int low, int high) {
```

```

    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1) return 1;

    int *arr = (int*)malloc(n * sizeof(int));

    srand(time(0));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 10000;
    }

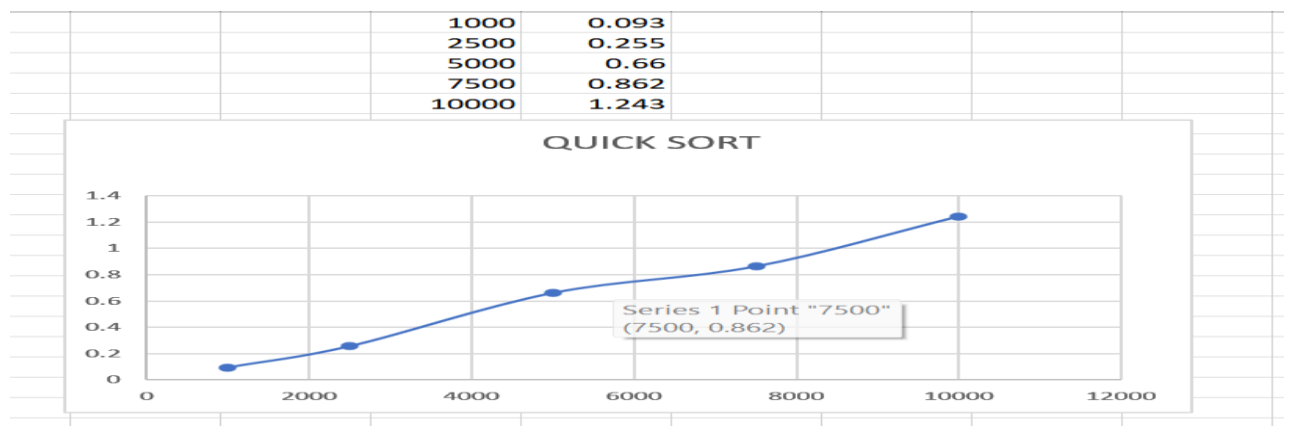
    clock_t start = clock();
    quickSort(arr, 0, n - 1);
    clock_t end = clock();

    double cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\nSorting complete.\n");
    printf("Time taken: %f seconds\n", cpu_time_used);
}

```

Output:



Output

Clear

Enter number of elements: 5

Enter 5 elements

51 46 79 64 31

Sorting complete.

31 46 51 64 79

Time taken: 0.000002 seconds

=== Code Execution Successful ===

Lab Program:5

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

Code:

```
#include <stdio.h>
#include <time.h>

void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && a[left] > a[largest])
        largest = left;

    if (right < n && a[right] > a[largest])
        largest = right;

    if (largest != i) {
        int temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;

        heapify(a, n, largest);
    }
}

void heapSort(int a[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(a, n, i);

    for (int i = n-1; i > 0; i--) {
        int temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

int main() {
```

```

int n;

printf("Enter number of elements: ");
scanf("%d", &n);

int a[n];

printf("Enter elements:\n");
for (int i = 0; i < n; i++)
    scanf("%d", &a[i]);

clock_t start, end;

start = clock();

heapSort(a, n);

end = clock();

printf("\nSorted Array:\n");
for (int i = 0; i < n; i++)
    printf("%d ", a[i]);

double time_taken =
    ((double)(end - start)) / CLOCKS_PER_SEC;

printf("\n\nTime Taken = %f seconds\n", time_taken);

return 0;
}

```

Output:

Output

Clear

Enter number of elements: 10

Enter elements:

6 15 76 8 0 57 12 90 24 1

Sorted Array:

0 1 6 8 12 15 24 57 76 90

Time Taken = 0.000002 seconds

=== Code Execution Successful ===

Lab Program:6

Implement 0/1 Knapsack problem using dynamic programming

Code:

```
#include <stdio.h>
int max(int a, int b) {
    return (a > b) ? a : b;
}
int main() {
    int n, W;
    printf("Enter number of items: ");
    scanf("%d", &n);

    int wt[n], val[n];
    printf("Enter weights:\n");

    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);
    printf("Enter values:\n");

    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    printf("Enter capacity: ");
    scanf("%d", &W);

    int dp[n+1][W+1];

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {

            if (i == 0 || w == 0)
                dp[i][w] = 0;

            else if (wt[i-1] <= w)
                dp[i][w] = max(
                    val[i-1] + dp[i-1][w - wt[i-1]],
                    dp[i-1][w]
                );
            else
                dp[i][w] = dp[i-1][w];
        }
    }
    printf("\nMaximum Value = %d\n", dp[n][W]);
    return 0;
}
```

Output:

```
Output Clear  
Enter number of items: 4  
Enter weights:  
50  
20  
30  
10  
Enter values:  
120  
200  
100  
110  
Enter capacity: 90  
  
Maximum Value = 430  
  
=== Code Execution Successful ===
```


LeetCode Problem:416

Given an integer array *nums*, return true *if you can partition the array into two subsets such that the sum of the elements in both subsets is equal* or false otherwise.

Code:

```
#include <stdbool.h>
#include <stdlib.h>

bool canPartition(int* nums, int numsSize) {
    int sum = 0;

    for (int i = 0; i < numsSize; i++) {
        sum += nums[i];
    }

    if (sum % 2 != 0)
        return false;

    int target = sum / 2;

    bool *dp = (bool *)calloc(target + 1, sizeof(bool));
    dp[0] = true;

    for (int i = 0; i < numsSize; i++) {
        for (int j = target; j >= nums[i]; j--) {
            dp[j] = dp[j] || dp[j - nums[i]];
        }
    }

    bool result = dp[target];
    free(dp);

    return result;
}
```

ScreenShot from LeetCode:

Description

Accepted

Editorial

Solutions

Submissions

← All Submissions

Accepted 147 / 147 testcases passed

Mandeep_singh_82 submitted at Jun 02, 2026 19:37

Analysis

Solution

Runtime

47 ms | Beats 50.78%

Memory

9.08 MB | Beats 39.91%



Code | C

```
1 #include <stdbool.h>
2 #include <stdlib.h>
3
4 bool canPartition(int* nums, int numsSize) {
5     int sum = 0;
6
7     for (int i = 0; i < numsSize; i++) {
8         sum += nums[i];
9     }
10
11     if (sum % 2 != 0)
12         return false;
13
14     int target = sum / 2;
15
16     bool *dp = (bool *)calloc(target + 1, sizeof(bool));
17     dp[0] = true;
18
19     for (int i = 0; i < numsSize; i++) {
20         for (int j = target; j >= nums[i]; j--) {
21             dp[j] = dp[j] || dp[j - nums[i]];
22         }
23     }
24
25     bool result = dp[target];
26     free(dp);
27
28     return result;
29 }
```

Code

```
1 #include <stdbool.h>
2 #include <stdlib.h>
3
4 bool canPartition(int* nums, int numsSize) {
5     int sum = 0;
6
7     for (int i = 0; i < numsSize; i++) {
8         sum += nums[i];
9     }
10
11     if (sum % 2 != 0)
12         return false;
13
14     int target = sum / 2;
15
16     bool *dp = (bool *)calloc(target + 1, sizeof(bool));
17     dp[0] = true;
18
19     for (int i = 0; i < numsSize; i++) {
20         for (int j = target; j >= nums[i]; j--) {
21             dp[j] = dp[j] || dp[j - nums[i]];
22         }
23     }
24
25     bool result = dp[target];
26     free(dp);
27
28     return result;
29 }
```

Saved

Ln 27, Col 1

Testcase

Test Result

Lab Program:7

Implement All Pair Shortest paths problem using Floyd's algorithm

Code:

```
#include <stdio.h>
#define MAX 100
#define INF 99999

void floyd(int graph[MAX][MAX], int n) {
    int dist[MAX][MAX];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            dist[i][j] = graph[i][j];

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    printf("\nAll-Pairs Shortest Path Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%3d ", dist[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int n, graph[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &n);
```

```
printf("Enter adjacency matrix (use 99999 for INF):\n");
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        scanf("%d", &graph[i][j]);

floyd(graph, n);

return 0;
}
```

Output:

Output Clear

Enter number of vertices: 5
Enter adjacency matrix (use 99999 for INF):
0 5 6 9999 2
5 0 9999 4 6
6 9999 0 3 5
9999 4 3 0 4
2 6 5 4 0

All-Pairs Shortest Path Matrix:
0 5 6 6 2
5 0 7 4 6
6 7 0 3 5
6 4 3 0 4
2 6 5 4 0

=== Code Execution Successful ===

Lab Program:8a

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm

Code:

```
#include <stdio.h>

#define MAX 10
#define INF 99999

int main() {
    int n;
    int graph[MAX][MAX];
    int selected[MAX] = {0};

    int edges = 0;
    int minCost = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix (0 for no edge):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    selected[0] = 1;

    printf("\nEdges in MST:\n");

    while (edges < n - 1) {

        int min = INF;
        int x = -1, y = -1;

        for (int i = 0; i < n; i++) {

            if (selected[i]) {

                for (int j = 0; j < n; j++) {

                    if (!selected[j] &&
                        graph[i][j] &&
                        graph[i][j] < min) {

                        min = graph[i][j];
                        x = i;
```

```

        y = j;
    }
}
}

printf("%d - %d : %d\n", x, y, graph[x][y]);

minCost += graph[x][y];
selected[y] = 1;
edges++;
}

printf("\nMinimum Cost = %d\n", minCost);

return 0;
}

```

Output:

Output

Clear

```

Enter number of vertices: 5
Enter adjacency matrix (0 for no edge):
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edges in MST:
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6

Minimum Cost = 16

=== Code Execution Successful ===

```

Lab Program:8b

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

Code:

```
#include <stdio.h>

#define MAX 10

struct Edge {
    int u, v, w;
};

int parent[MAX];

// Find parent
int find(int i) {
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void unite(int a, int b) {
    parent[a] = b;
}

void sort(struct Edge e[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (e[j].w > e[j + 1].w) {
                struct Edge temp = e[j];
                e[j] = e[j + 1];
                e[j + 1] = temp;
            }
        }
    }
}

int main() {
    int n;
    int graph[MAX][MAX];
    struct Edge edges[MAX * MAX];

    int edgeCount = 0;
    int minCost = 0;
```

```

printf("Enter number of vertices: ");
scanf("%d", &n);

printf("Enter adjacency matrix (0 for no edge):\n");

for (int i = 0; i < n; i++) {
    parent[i] = i;

    for (int j = 0; j < n; j++) {
        scanf("%d", &graph[i][j]);

        if (graph[i][j] != 0 && i < j) {
            edges[edgeCount].u = i;
            edges[edgeCount].v = j;
            edges[edgeCount].w = graph[i][j];
            edgeCount++;
        }
    }
}

sort(edges, edgeCount);

printf("\nEdges in MST:\n");

int count = 0;

for (int i = 0; i < edgeCount && count < n - 1; i++) {

    int u = find(edges[i].u);
    int v = find(edges[i].v);

    if (u != v) {
        printf("%d - %d : %d\n",
            edges[i].u,
            edges[i].v,
            edges[i].w);

        minCost += edges[i].w;
        unite(u, v);
        count++;
    }
}

printf("\nMinimum Cost = %d\n", minCost);

return 0;
}

```


Output:

Output

Clear

```
Enter number of vertices: 4
Enter adjacency matrix (0 for no edge):
0 2 0 6
2 0 3 8
0 3 0 5
6 8 5 0
```

Edges in MST:

```
0 - 1 : 2
1 - 2 : 3
2 - 3 : 5
```

Minimum Cost = 10

=== Code Execution Successful ===

Lab Program:9

Implement Fractional Knapsack using Greedy technique

Code:

```
#include <stdio.h>

struct Item {
    int weight, value;
    float ratio;
};

void sort(struct Item arr[], int n) {
    for (int i = 0; i < n-1; i++)
        for (int j = 0; j < n-i-1; j++)
            if (arr[j].ratio < arr[j+1].ratio) {
                struct Item temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
}

int main() {
    int n, capacity;
    struct Item arr[100];

    printf("Enter number of items: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("\nItem %d\n", i+1);
        printf("Value: ");
        scanf("%d", &arr[i].value);
        printf("Weight: ");
        scanf("%d", &arr[i].weight);
        arr[i].ratio = (float)arr[i].value / arr[i].weight;
    }

    printf("\nEnter knapsack capacity: ");
    scanf("%d", &capacity);

    sort(arr, n);

    float totalValue = 0.0;

    printf("\nItems taken:\n");
```

```

for (int i = 0; i < n; i++) {
    if (capacity == 0) break;

    if (arr[i].weight <= capacity) {

        capacity -= arr[i].weight;
        totalValue += arr[i].value;
        printf("Item %d -> FULL (Value = %d)\n", i+1, arr[i].value);
    } else {

        float fraction = (float)capacity / arr[i].weight;
        totalValue += arr[i].value * fraction;
        printf("Item %d -> %.2f fraction\n", i+1, fraction);
        capacity = 0;
    }
}

printf("\nMaximum Value = %.2f\n", totalValue);

return 0;
}

```

Output:

Output

Clear

```

Enter number of items: 5

Item 1
Value: 50
Weight: 50

Item 2
Value: 30
Weight: 10

Item 3
Value: 60
Weight: 70

Item 4
Value: 10
Weight: 5

Item 5
Value: 90
Weight: 25

Enter knapsack capacity: 120

Items taken:
Item 1 -> FULL (Value = 90)
Item 2 -> FULL (Value = 30)
Item 3 -> FULL (Value = 10)
Item 4 -> FULL (Value = 50)
Item 5 -> 0.43 fraction

Maximum Value = 205.71

=== Code Execution Successful ===

```

LeetCode Problem:11

You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the i^{th} line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return *the maximum amount of water a container can store*.

Notice that you may not slant the container.

Code:

```
int maxArea(int* height, int heightSize) {
    int left = 0;
    int right = heightSize - 1;
    int maxWater = 0;

    while (left < right) {
        int width = right - left;

        int h = (height[left] < height[right]) ? height[left] : height[right];

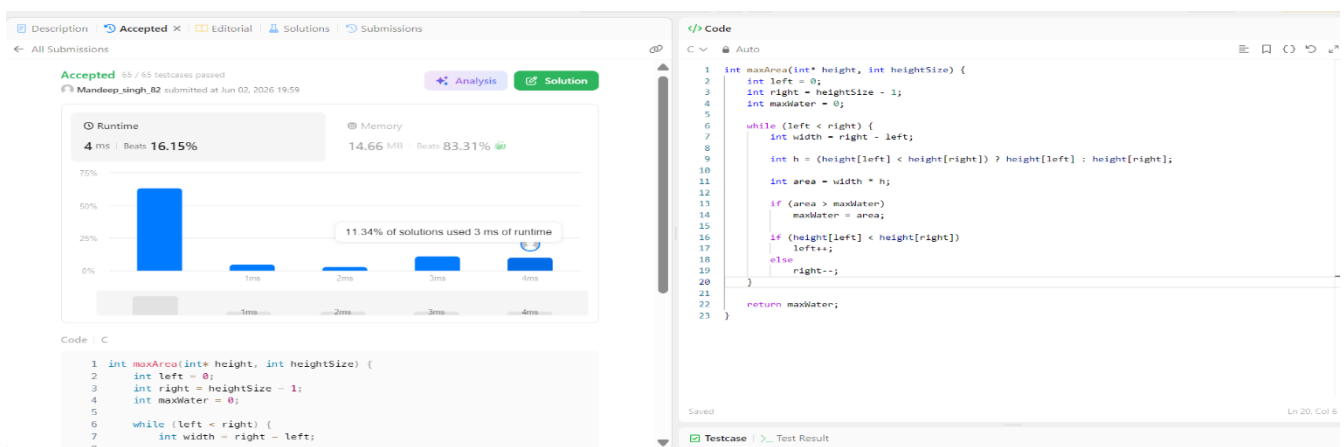
        int area = width * h;

        if (area > maxWater)
            maxWater = area;

        if (height[left] < height[right])
            left++;
        else
            right--;
    }

    return maxWater;
}
```

Output:



Lab Program:10

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm

Code:

```
#include <stdio.h>
#define MAX 100
#define INF 99999

int minDist(int dist[], int visited[], int n) {
    int min = INF, index = -1;

    for (int i = 0; i < n; i++) {
        if (!visited[i] && dist[i] < min) {
            min = dist[i];
            index = i;
        }
    }
    return index;
}

void dijkstra(int graph[MAX][MAX], int n, int src) {
    int dist[MAX], visited[MAX];

    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }
    dist[src] = 0;

    for (int i = 0; i < n-1; i++) {
        int u = minDist(dist, visited, n);
        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (!visited[v] && graph[u][v] &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("\nVertex\tDistance from Source\n");
```

```

        for (int i = 0; i < n; i++)
            printf("%d\t%d\n", i, dist[i]);
    }

int main() {
    int n, src;
    int graph[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix (0 for no edge):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    printf("Enter source vertex: ");
    scanf("%d", &src);

    dijkstra(graph, n, src);

    return 0;
}

```

Output:

Output

Clear

```

Enter number of vertices: 5
Enter adjacency matrix (0 for no edge):
0 5 6 0 2
5 0 0 4 6
6 0 0 3 5
0 4 3 0 4
2 6 5 4 0
Enter source vertex: 0

Vertex  Distance from Source
0      0
1      5
2      6
3      6
4      2

```

=== Code Execution Successful ===

Lab Program:11

Implement “N-Queens Problem” using Backtracking

Code:

```
#include <stdio.h>
#include <stdlib.h>

int x[20], n;

int place(int k, int i) {
    for (int j = 1; j < k; j++)
        if (x[j] == i || abs(x[j] - i) == abs(j - k))
            return 0;
    return 1;
}

void queen(int k) {
    for (int i = 1; i <= n; i++) {
        if (place(k, i)) {
            x[k] = i;
            if (k == n) {
                for (int j = 1; j <= n; j++)
                    printf("%d ", x[j]);
                printf("\n");
            } else
                queen(k + 1);
        }
    }
}

int main() {
    printf("Enter n: ");
    scanf("%d", &n);
    queen(1);
    return 0;
}
```

Output:

Output

Clear

Enter n: 5

1 3 5 2 4

1 4 2 5 3

2 4 1 3 5

2 5 3 1 4

3 1 4 2 5

3 5 2 4 1

4 1 3 5 2

4 2 5 3 1

5 2 4 1 3

5 3 1 4 2

=== Code Execution Successful ===

LeetCode Problem:704

Given an array of integers nums which is sorted in ascending order, and an integer target, write a function to search target in nums. If target exists, then return its index. Otherwise, return -1.

You must write an algorithm with $O(\log n)$ runtime complexity.

Code:

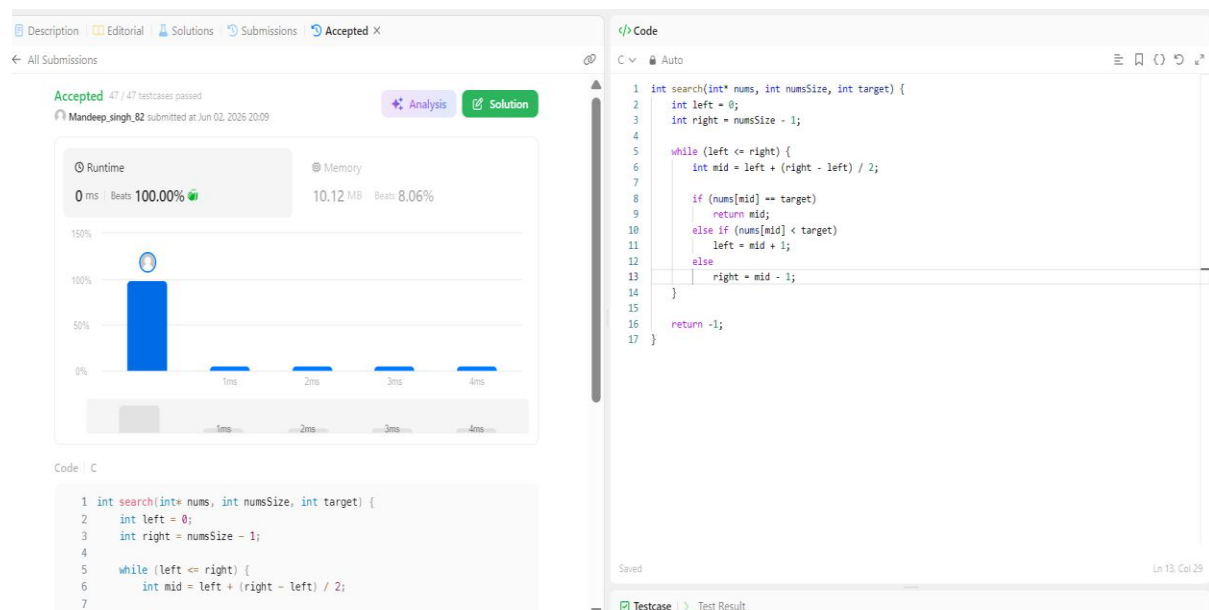
```
int search(int* nums, int numsSize, int target) {
    int left = 0;
    int right = numsSize - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target)
            return mid;
        else if (nums[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }

    return -1;
}
```

ScreenShot from LeetCode:



LeetCode Problem:268

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return *the only number in the range that is missing from the array.*

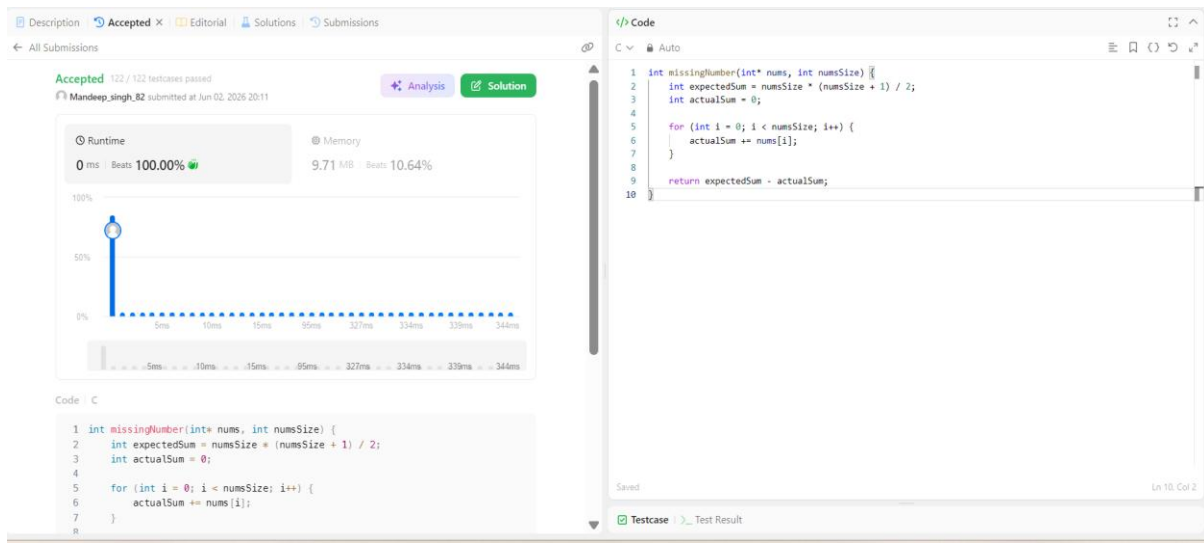
Code:

```
int missingNumber(int* nums, int numsSize) {
    int expectedSum = numsSize * (numsSize + 1) / 2;
    int actualSum = 0;

    for (int i = 0; i < numsSize; i++) {
        actualSum += nums[i];
    }

    return expectedSum - actualSum;
}
```

ScreenShot from LeetCode:



LeetCode Problem:1636

Given an array of integers `nums`, sort the array in **increasing** order based on the frequency of the values. If multiple values have the same frequency, sort them in **decreasing** order. Return the *sorted array*.

Code:

```
#include <stdlib.h>

typedef struct {
    int value;
    int freq;
} Pair;

int compare(const void *a, const void *b) {
    Pair *p1 = (Pair *)a;
    Pair *p2 = (Pair *)b;

    if (p1->freq != p2->freq)
        return p1->freq - p2->freq;    // increasing frequency

    return p2->value - p1->value;    // decreasing value
}

int* frequencySort(int* nums, int numsSize, int* returnSize) {
    Pair arr[numsSize];

    for (int i = 0; i < numsSize; i++) {
        int count = 0;
        for (int j = 0; j < numsSize; j++) {
            if (nums[i] == nums[j])
                count++;
        }
        arr[i].value = nums[i];
        arr[i].freq = count;
    }
    qsort(arr, numsSize, sizeof(Pair), compare);

    int *result = (int *)malloc(numsSize * sizeof(int));

    for (int i = 0; i < numsSize; i++) {
        result[i] = arr[i].value;
    }

    *returnSize = numsSize;
    return result;
}
```

ScreenShot from LeetCode:

